FH University of Applied Sciences TECHNIKUM WIEN

Protocol

Tour Planner

Erstellt von: Vuksan Bozo, Gröstenberger Jaspher
Studiengang: Bachelor Informatik 4
Lehrveranstaltung: Softwareentwicklung 2
LektorIn: Michael Hlobil

Wien am, 29. Jun. 2026

Inhaltsverzeichnis

Inhalt

1. The idea

"Tour Planner" is a private, web-based digital logbook designed for outdoor enthusiasts to plan trips and systematically track their physical achievements. The application allows users to create customized routes for hiking, biking, or running, featuring automated distance calculations and interactive maps. By attaching multiple activity logs to a single tour, users can monitor their personal progress over time across the exact same routes.

2. Requirements Overview

- **User Authentication:** Secure self-registration and login with strict single-user data isolation (no cross-user sharing).
- **Tour Management (CRUD):** Full management of tours, utilizing the **OpenRouteservice.org API** for route calculation and **Leaflet** for graphical map rendering.
- **Tour Logs (CRUD):** A 1:N relationship allowing users to log specific trip statistics (date, difficulty, time, rating, comments) for each tour.
- **Analytics:** Automatic background computation of tour metrics, specifically *popularity* (based on log count) and *child-friendliness* (derived from distance and difficulty).
- **Search & Data Portability:** Universal full-text search considering all raw and computed values, strict input validation, data import/export functionality, and a custom unique feature (e.g., PDF report generation).

3. Goals

- **Clean Frontend Architecture:** Develop a responsive interface in **Angular** adhering strictly to the **MVVM pattern** and containing at least one custom, reusable web component.
- **Layered Backend Structure:** Implement a client-server system using **ASP.NET Core (C#)** split into three distinct layers: Presentation, Business Logic (BL), and Data Access (DAL), incorporating established software design patterns.
- **Data Persistence:** Store relational tour data via Entity Framework Core in a **PostgreSQL** database, while keeping image files separate on the server's filesystem.
- **Quality & Configuration:** Integrate a robust logging framework, achieve test coverage using **NUnit**, externalize all environmental configurations (e.g., DB connection strings), and fully document the system using UML diagrams and wireframes.

4. UI/UX concept

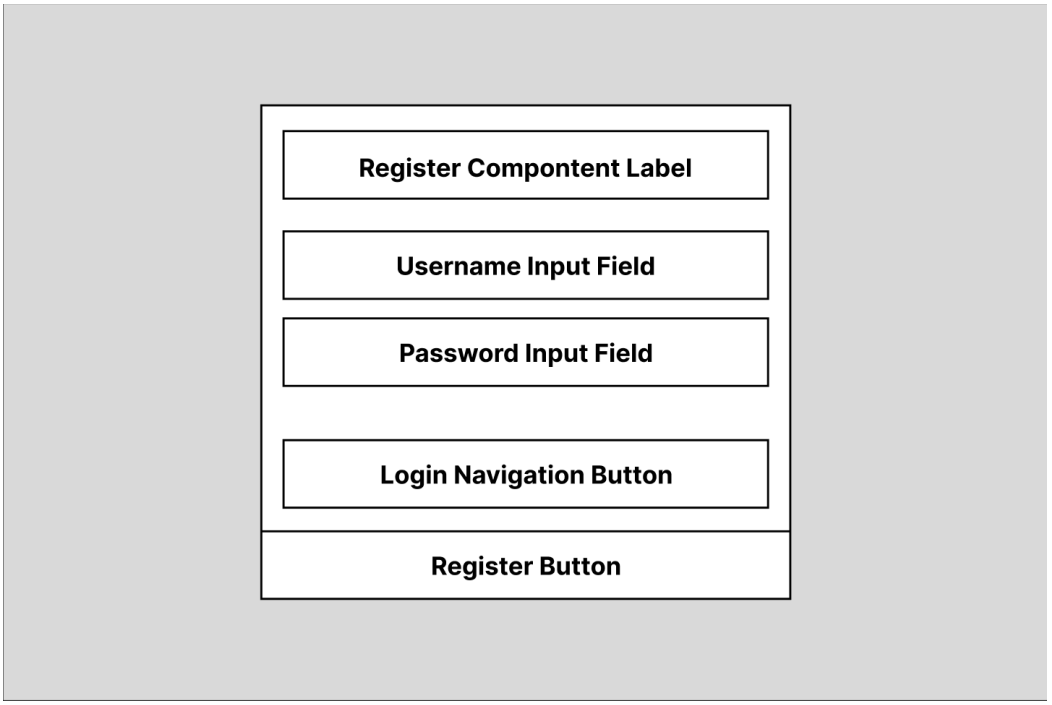


Figure 1: Registration View

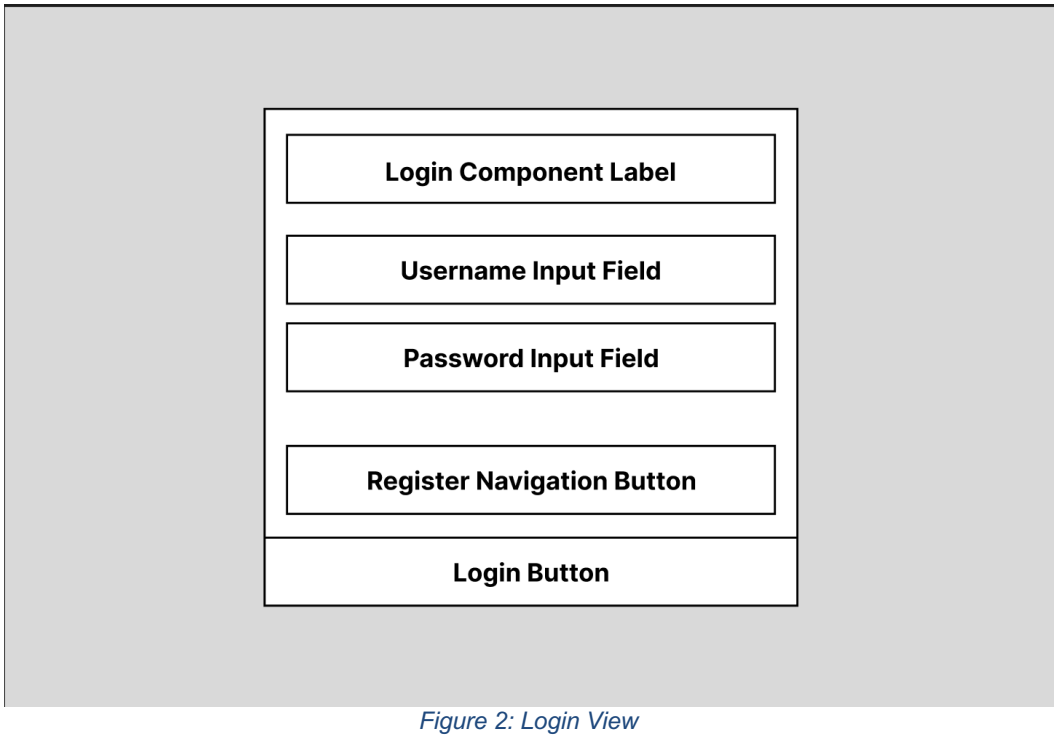


Figure 2: Login View

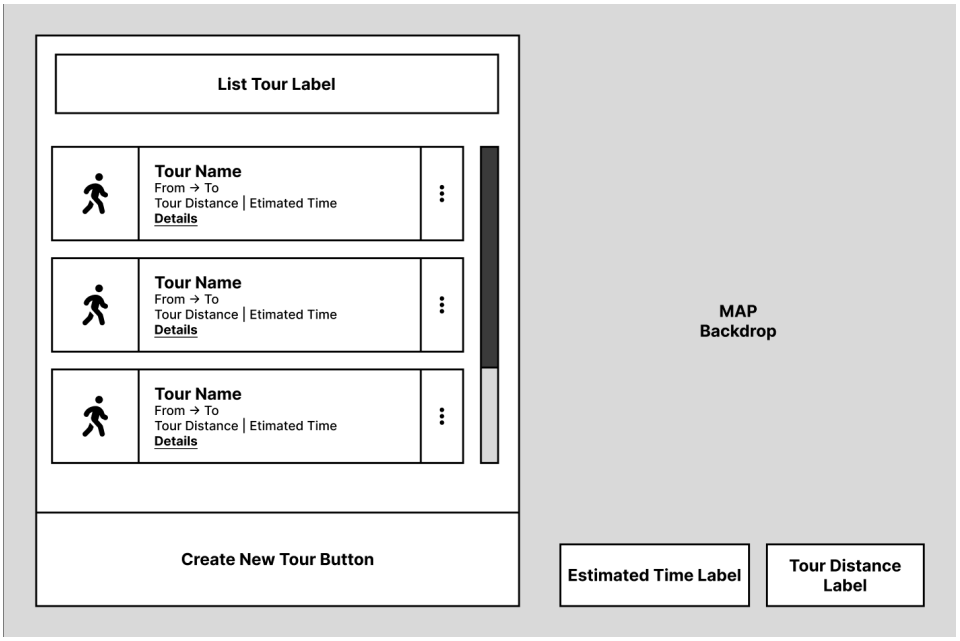


Figure 3: Tour List View

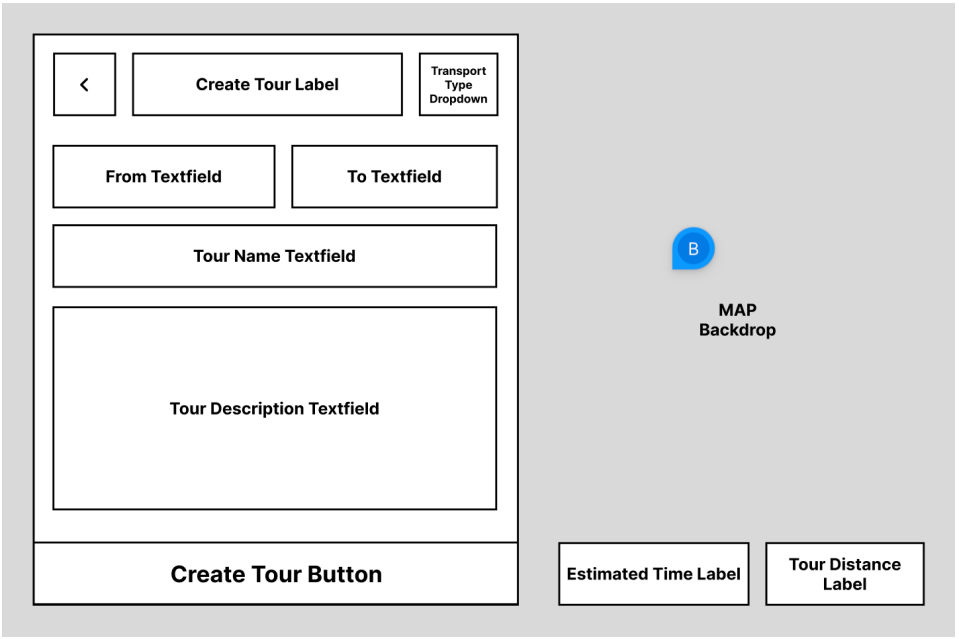


Figure 4: Create Tour View

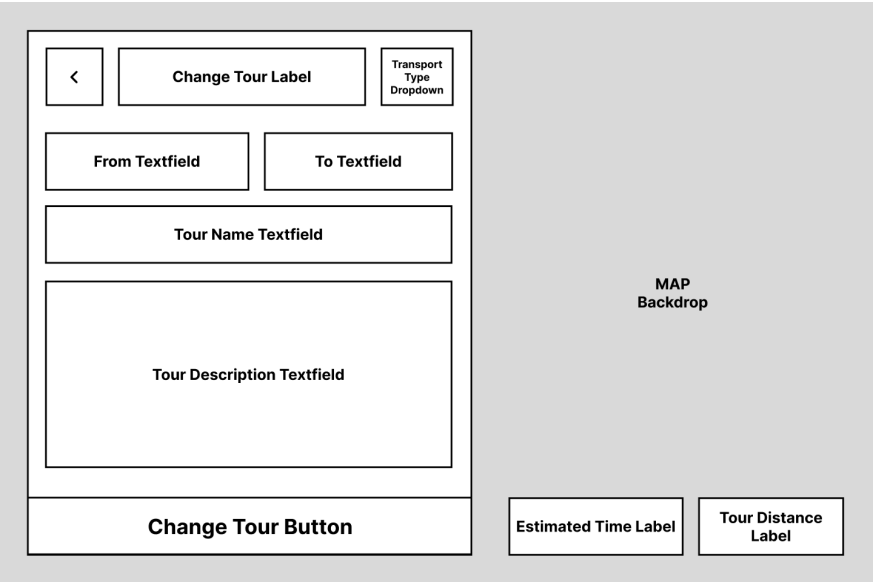


Figure 5: Update Tour View

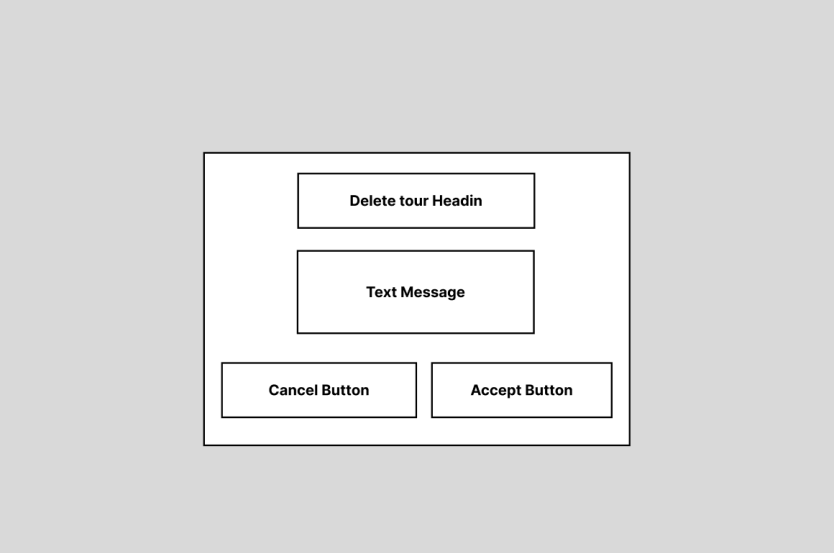


Figure 6: Delete Tour View

5. Architecture and Design Patterns

Layered Architecture

Presentation Layer
Contents
- UserController
- ToursController
- LogsController

Functionality

The "outer layer" (or facade) of the application. It handles incoming HTTP requests, validates JWT tokens, and returns appropriate HTTP responses (e.g., `Ok`, `BadRequest`). It remains entirely decoupled from the database implementation.

Business Logic Layer
Contents
- UserService
- UserAlreadyExistsException
- UserNotFoundException
- TourService
- TourLogService

Functionality

The "brain" of the application, housing the core business logic. This layer handles data processing, executes business validation rules (e.g., ensuring a distance value is positive), and utilizes `IMapper` to map internal domain models to Data Transfer Objects (DTOs).

Data Access Layer
Contents
- AppDBC
- BaseRepository
- UserRepository
- TourRepository
- TourLogRepository

Functionality

The "data provider" (or data supplier). It interacts directly with the PostgreSQL database using EF Core. This layer is responsible for handling all CRUD operations (fetching, saving, updating, or deleting) on the raw data.

Model
Contents
- User
- Tour
- TourLog

Functionality

The common data structures utilized across all layers to transfer and pass data throughout the application.

Design Patterns - Frontend

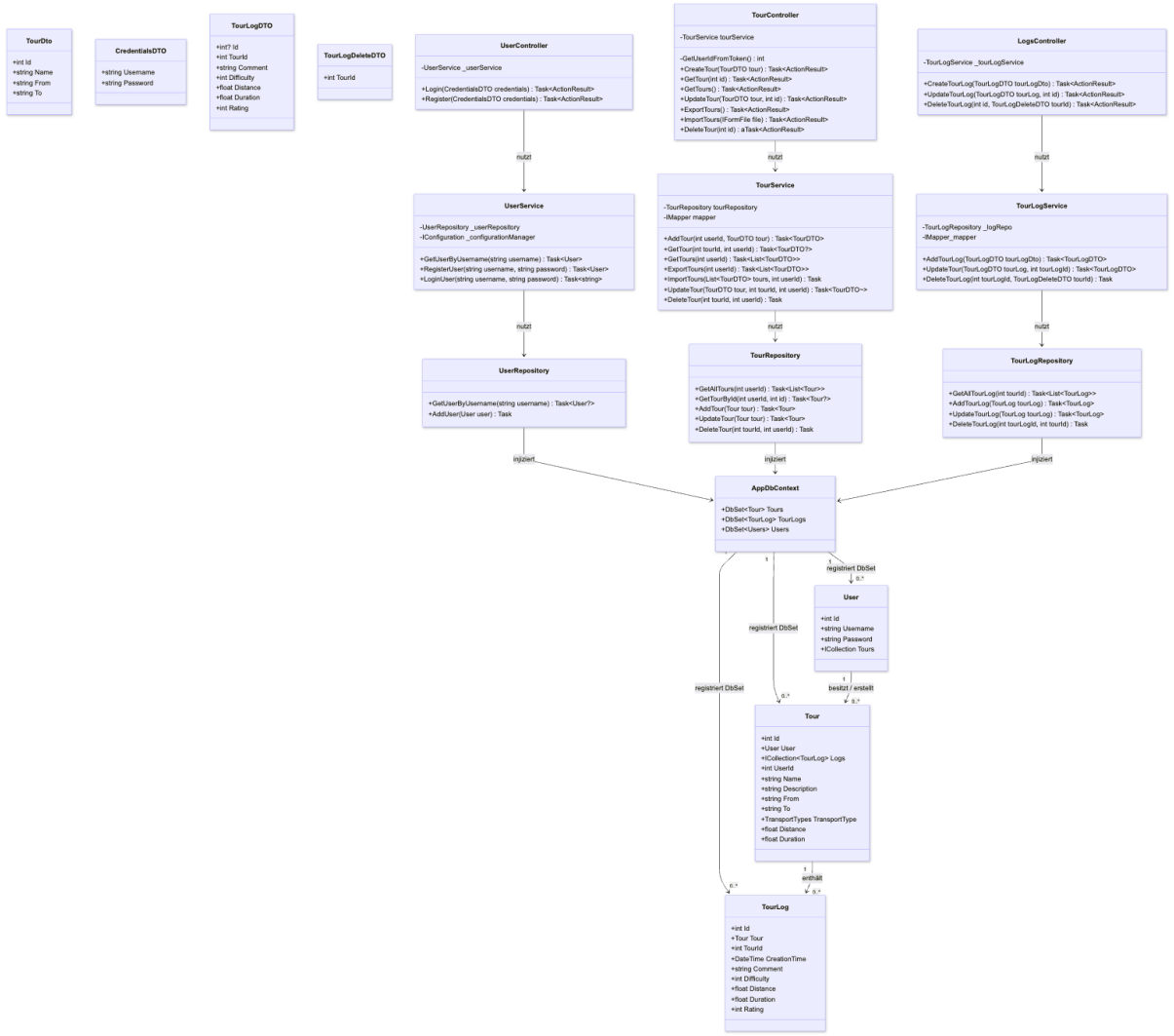
Facade Pattern

We used the **Facade Pattern** by creating the MapService to make working with the Leaflet library much easier. Instead of forcing our Angular components to deal with complex Leaflet logic—like parsing GeoJSON data, setting up custom markers, or calculating map boundaries—the service hides all these details. It provides simple, easy-to-use methods like setRoute() or moveTo(). This keeps our components clean, lightweight, and completely decoupled from the map library's specific code.

Singleton Pattern

We utilized the **Singleton Pattern** by leveraging Angular's Dependency Injection (inject()) framework. Key components like the TourStore (which manages the global state of our tours) and central services such as MapService and TourService are registered as root-level singletons. This ensures that only one single instance of these classes exists across the entire application. As a result, our TourCRUD component interacts with the exact same data store and map state as the rest of the application, avoiding redundant instances and conserving system resources.

UML Class Diagramm - Backend



6. Libraries

Log4Net

For logging and error tracking, we integrated the **log4net** library into our application. It provides a robust and flexible logging framework that allows us to log messages at different severity levels (like INFO, WARN, and ERROR). By configuring log4net, we can easily capture runtime behavior and errors, and direct the output to specific targets like files or the console. This helps us monitor the application during development and makes debugging issues much easier.

AutoMapper

To simplify data transfer between different layers of our application, we used the **AutoMapper** library. In our backend, we often need to convert database entities into Data Transfer Objects (DTOs), like the TourRequestDTO used in our frontend. Instead of writing tedious, repetitive mapping code by hand for every single property, AutoMapper handles this automatically based on name matching. This significantly reduces boilerplate code, minimizes human error during object conversion, and keeps our controllers and services clean and readable.

NUnit

To run our automated tests, we chose **NUnit** as our core testing framework. NUnit provides the structure for our test suite, allowing us to define test cases using simple attributes like [Test] or [SetUp]. It also gives us a wide range of assertions to check if our code actually behaves the way we expect. Combined with NSubstitute, NUnit forms the foundation of our testing environment, ensuring that any bugs or regressions are caught automatically during development before the code goes live.

NSubstitute

For unit testing our backend logic, we used **NSubstitute** as our mocking framework. When testing our services or controllers, we often need to isolate the class under test from its dependencies, such as the database or external APIs. NSubstitute allows us to easily create "substitute" (mock) objects that mimic these dependencies. We can define exactly how these mocks should behave and verify that our code calls the correct methods with the expected arguments. This makes our unit tests fast, reliable, and focused purely on the logic we actually want to test.

7. Use-Cases

User Authentication (Login)

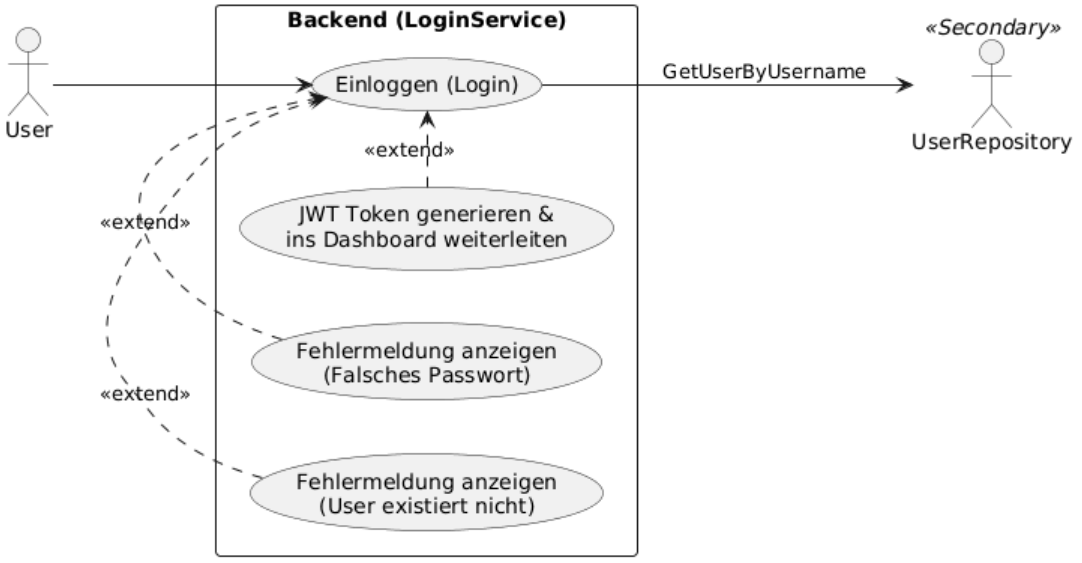


Figure 7: Use-case diagram; UserAuth.

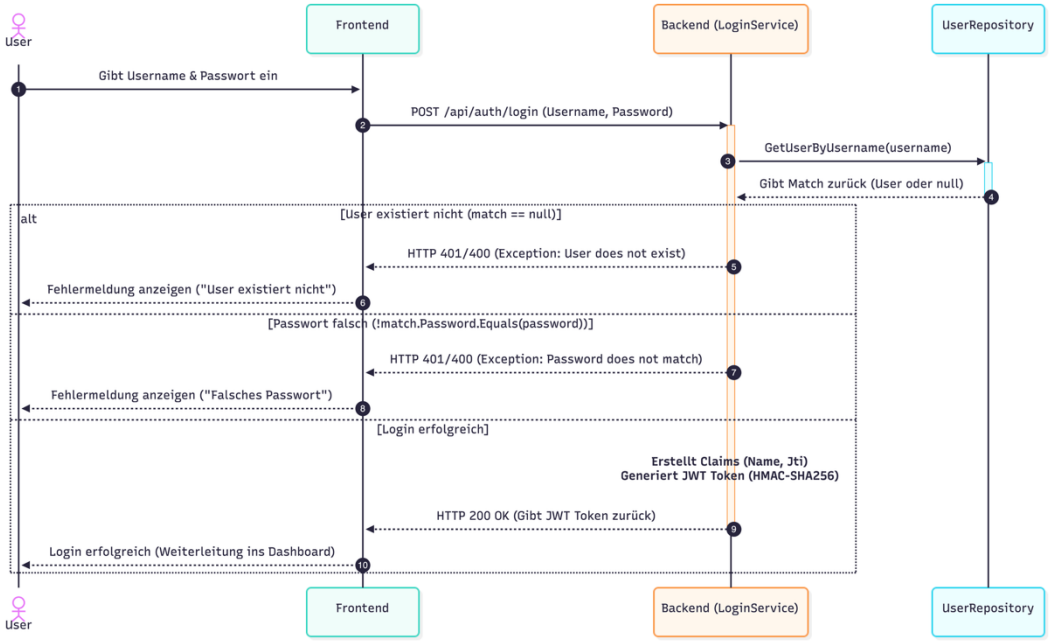


Figure 8: Sequence diagram; UserAuth.

The system provides a secure authentication mechanism handled by the **Backend (LoginService)**. The main goal of this interaction is to verify user credentials and issue a secure Access Token (JWT) upon successful authentication.

The authentication process is initiated by the **User** (Primary Actor), who submits their credentials via the user interface. The Frontend forwards this payload as a POST request to the LoginService. To validate the request, the backend coordinates with the **UserRepository** (Secondary Actor) to fetch the stored user entity by their username.

Depending on the state of the data store and the provided input, the control flow diverges into three mutually exclusive scenarios within the system boundary:

- **Scenario A (User Missing):** If the UserRepository returns no match (null), the service stops further execution, logs a bad request/unauthorized exception, and triggers an extension flow that instructs the client to display the error *"User existiert nicht"*.
- **Scenario B (Invalid Credentials):** If the user is found but the password verification fails against the stored hash, the system aborts execution, generates an unauthorized response, and triggers an extension flow to display the error *"Falsches Passwort"*.
- **Scenario C (Successful Authentication):** If both checks pass, the core service generates user claims and signs a secure **JWT Token** using HMAC-SHA256. This token is returned via an HTTP 200 OK status code, granting the user access and triggering a redirection to the application dashboard.

Tour Management (Add Tour)

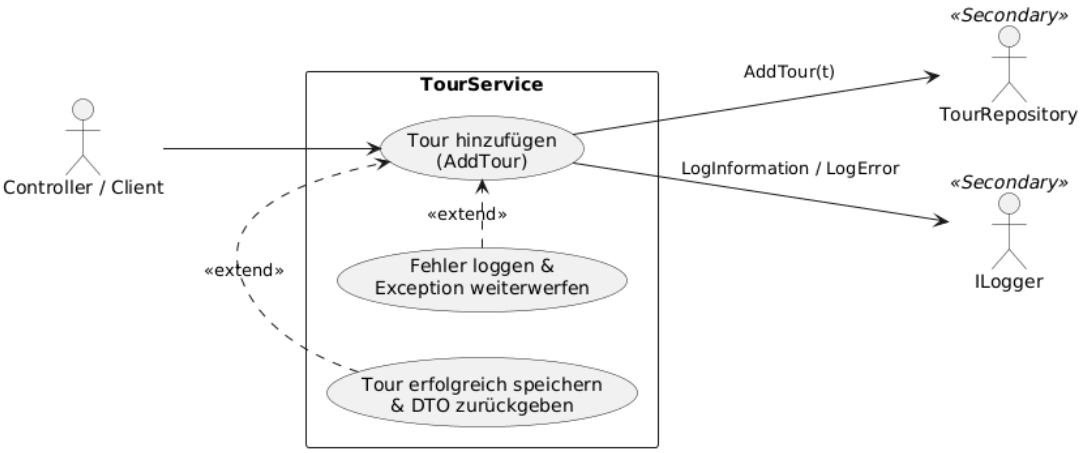


Figure 9: Use-case diagram; add tour

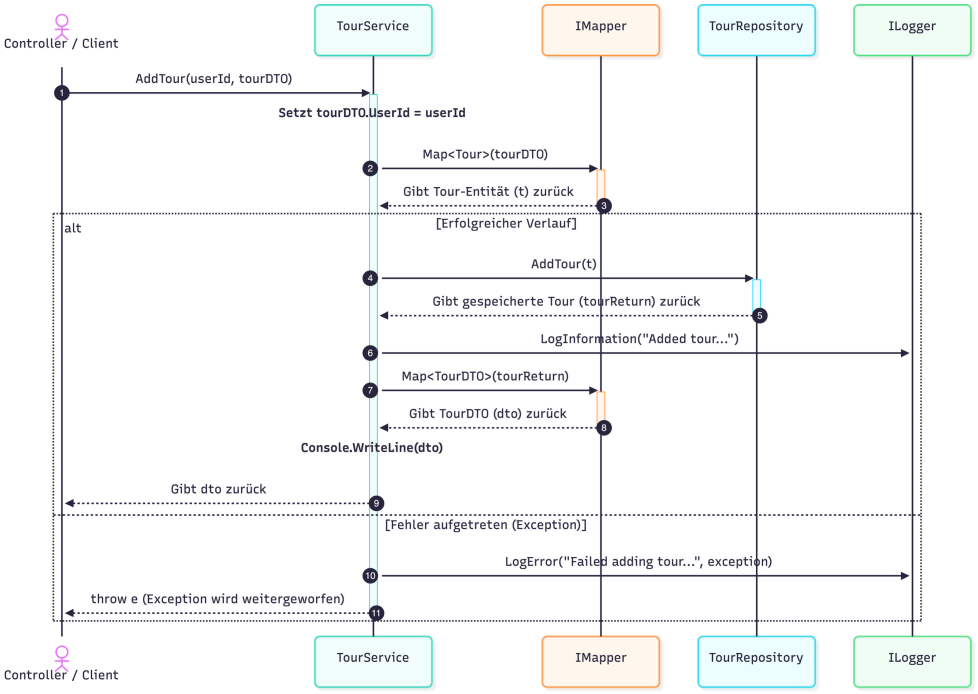


Figure 10: Sequence diagram; add tour

The system provides a business logic layer encapsulated by the **TourService** to handle the creation and persistence of tours. The primary objective is to process incoming data transfer objects (DTOs), map them to database entities, and save them securely while ensuring full operational logging

The workflow is initiated by the **Controller / Client** (Primary Actor), triggering the AddTour(userId, tourDTO) routine. Upon receiving the payload, the TourService internally utilizes an IMapper component to transform the data contract into a domain-specific tour entity (t).

Following the mapping process, a conditional alt block defines two mutually exclusive execution branches depending on the transaction state:

- **Scenario A (Successful Path):** The service calls the **TourRepository** (Secondary Actor) to persist the entity via AddTour(t). After receiving the confirmation, the event is recorded by calling LogInformation on the **ILogger**(Secondary Actor). Finally, the persisted entity is converted back into a data contract and returned to the client.
- **Scenario B (Error Path / Exception):** If an exception occurs at any point during execution, the service catches the failure, invokes LogError on the **ILogger** to preserve the stack trace, and immediately rethrows (throw e) the exception back to the calling controller to prevent silent failures.

Tour Log Management (Update Tour Log)

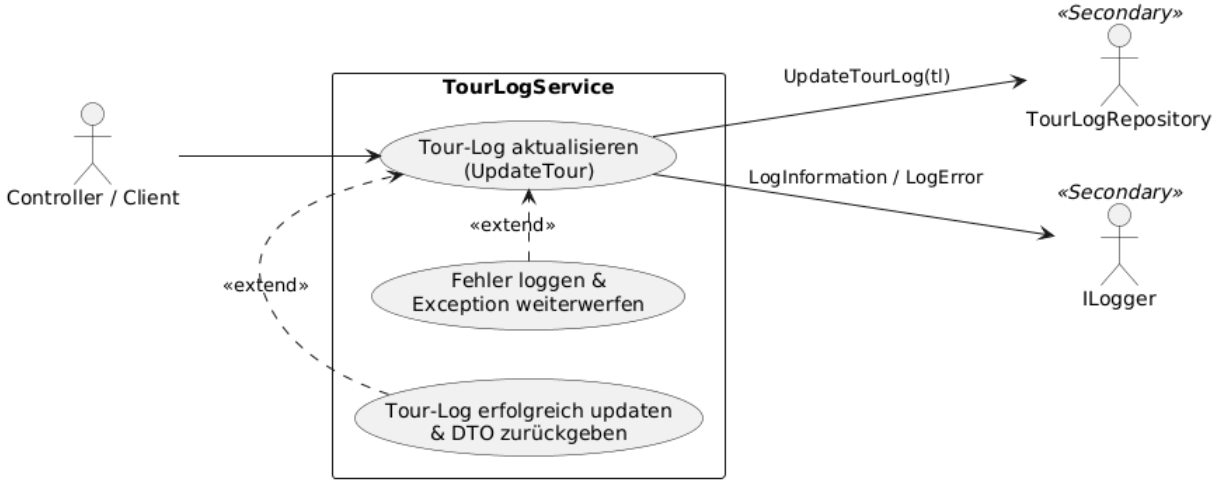


Figure 11: Use-case diagram; update tour log

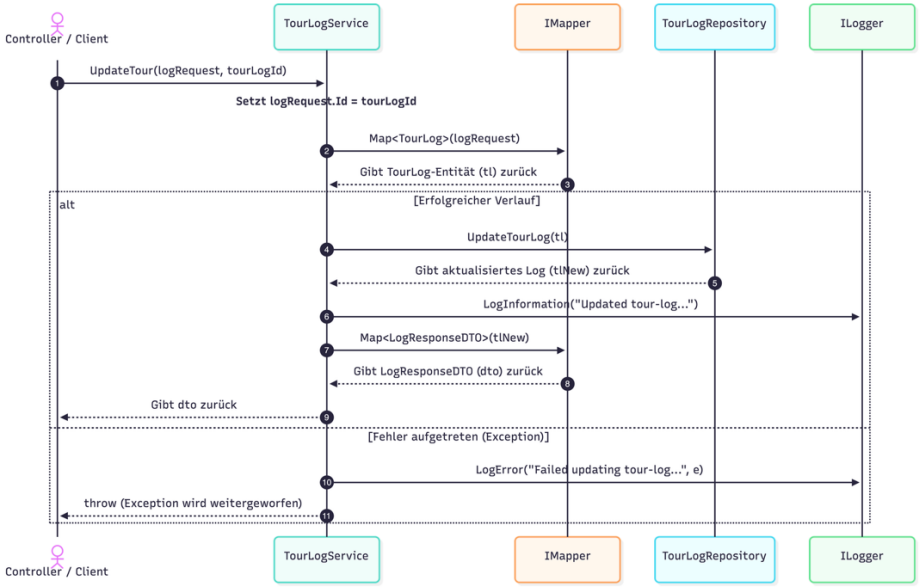


Figure 12: Sequence diagram; update tour log

The system provides a specific business logic component encapsulated by the **TourLogService** to handle updates to existing tour logs. Its primary objective is to map incoming request payload modifications to domain entities, synchronize those updates with the persistent storage layer, and provide diagnostics through systemic logging.

The orchestration begins when the **Controller / Client** (Primary Actor) invokes the `UpdateTour(logRequest, tourLogId)` interface method. The `TourLogService` updates the identifier metadata and passes the payload to an internal `IMapper` instance to translate the request model into a concrete `TourLog` domain entity (`tl`).

The runtime path then diverges inside a conditional `alt` block based on the stability of downstream interactions:

- **Scenario A (Successful Path):** The transaction proceeds by calling the **TourLogRepository** (Secondary Actor) via `UpdateTourLog(tl)`. Once the updated state (`tlNew`) is safely returned, an operational checkpoint is recorded via the **ILogger** (Secondary Actor) using `LogInformation`. The fresh state is mapped into a `LogResponseDTO` and dispatched back to the caller.
- **Scenario B (Error Path / Exception):** If an infrastructural or validation exception interrupts the update workflow, control passes to the failure block. The service calls `LogError` on the **ILogger** to capture the exact contextual exception details, then preserves transparency by rethrowing (`throw e`) the unmodified error back upstream to the client layer.

8. Testing Decisions

1. Why the Tested Code is Critical (Business Logic Layer)

The tested classes (UserService, TourService, TourLogService) form the Business Logic Layer (BLL). This layer is the core of the application and is critical for three main reasons:

- **Security & Access Control:** It acts as the gatekeeper, generating secure JWT tokens, validating passwords, and ensuring users can only access or modify their own data.
- **Data Integrity:** It cleanses and validates incoming Data Transfer Objects (DTOs) from the frontend, preventing malicious payload manipulation before data reaches the database.
- **Orchestration:** It securely manages the mapping between DTOs and database entities (using AutoMapper) and enforces business rules.

2. Rationale for Test Selection

The 20+ unit tests were strategically chosen to cover not just "happy paths," but primarily error handling, security, and edge cases:

- **UserService (Authentication & Authorization):** Tests ensure that valid login attempts generate correct JWT tokens. More importantly, they verify that duplicate registrations are prevented and invalid credentials rigorously trigger the correct exceptions, preventing unauthorized access.
- **TourService (Core Data Flow & Edge Cases):** Tests validate crucial data binding, such as proving the system securely sets the UserId on the backend rather than trusting the user payload. Furthermore, tests for the import logic ensure that existing IDs are nullified to prevent accidental database overwrites. Edge cases (e.g., querying non-

existent IDs or empty lists) are tested to guarantee predictable application behavior without crashes.

- **TourLogService (Manipulation Prevention):** These tests focus heavily on relational data integrity. A critical security test simulates a malicious attempt to alter a TourLog ID via the JSON payload. The test proves that the service ignores the payload ID and strictly overwrites it with the legitimate URL parameter, successfully neutralizing the attack.

Conclusion By mocking the Data Access Layer (DAL), these tests isolate the BLL to prove that all business rules, data mappings, and security mechanisms function reliably and independently of the database state.

9. Git Link

https://github.com/Vukbo/TP

10. Tracked time

~40+ hrs for entire projects